Pharmacy Manager

Static Code Analysis Report, Issues & Reflection

1. Objective

The objective of this lab was to analyse the Pharmacy Manager Python program using Pylint, Bandit, and Flake8, document the issues, apply corrective changes, and verify that the program continued to perform its intended demonstration tasks.

2. Tools and Results

Tool	Focus	Before	Final recorded result
Pylint	Code quality, maintainability, common mistakes	4.83/10	10.00/10
Bandit	Security-sensitive Python patterns	8 findings (4 low, 2 medium, 2 high)	0 findings
Flake8	Style and formatting	21 violations	0 violations

These are the results recorded during the lab. Re-run all three tools on the exact final checkout if the source code has changed since these reports were generated.

3. Issues Identified and Fixes

Issue	Type / severity	Description	Fix approach
Unsafe eval() for formula evaluation	High — security	Untrusted expressions could execute arbitrary Python code.	Use a restricted AST-based arithmetic evaluator that permits only approved nodes and numeric values.
Pickle used for stock data	High — security	Loading an untrusted pickle can execute code.	Use JSON and validate the loaded structure and values.
MD5 for patient identifiers	High — cryptography	MD5 is not suitable for security-sensitive identifiers.	Use HMAC-SHA-256 with a secret supplied via an environment variable.
Hardcoded pharmacist password	High — credentials	Credentials embedded in source can be exposed through repository access.	Read the demo password from an environment variable.
os.system() for label printing	Medium — command execution	Shell invocation creates avoidable command-injection risk.	Print directly in Python for the demonstration.
random.randint() for prescription number	Medium — randomness	The random module is not intended for security-sensitive or unpredictable values.	Use secrets for a less predictable demo identifier; use persistent uniqueness if required.
Broad or bare exception handling	Medium — error handling	Broad handling can hide errors and make debugging difficult.	Catch expected exceptions and report actionable errors.
Mutable default audit=[]	Low/medium — correctness	A shared list can leak entries across function calls.	Use None as the default and create a fresh list inside the function.
Missing docstrings and naming inconsistencies	Low — maintainability	Poor documentation and inconsistent naming reduce readability.	Add docstrings and use snake_case consistently.
Unused imports and formatting violations	Low — style	Unused imports, long lines, whitespace and newline issues reduce maintainability.	Remove unused imports, wrap long lines, and normalise whitespace.

Line-number note: exact line numbers vary as code changes. Use the original snapshot and `nl -ba pharmacy_manager_original.py` to identify precise original line numbers if required by the instructor.

4. Fixes and Functional Verification

• Replaced unrestricted expression evaluation with restricted arithmetic parsing.

• Replaced pickle stock serialization with JSON and validation.

• Replaced MD5-based identifiers with HMAC-SHA-256 using an environment-configured secret.

• Moved the demo pharmacist password to an environment variable.

• Removed shell invocation for label printing.

• Used secrets for prescription-number generation.

• Improved exception handling and corrected the mutable default argument.

• Improved documentation, naming, imports, and formatting.

The corrected program was run successfully during the lab. Demonstrated behaviour included stock reporting, an expired-product check, billing, dosage, login, and stock updates. Static analysis complements functional testing; it does not prove that a program is entirely defect-free.

5. Reflection

5.1 What did you learn from using static analysis tools?

I learned that the tools identify different categories of problems. Pylint focuses on code quality, maintainability, naming, documentation, and common programming mistakes. Bandit highlights security-sensitive patterns such as eval(), unsafe pickle loading, weak hashing, hardcoded credentials, shell execution, and unsuitable random-number generation. Flake8 checks Python style and formatting. Using the tools together gave a broader review than relying on program execution alone.

5.2 Which issues were most important, and why?

The highest-priority issues were eval(), pickle, the hardcoded password, and MD5-based patient identifiers. Depending on how input and data files are handled, these can create code-execution, credential-exposure, or cryptographic risks. The os.system() call and use of random for prescription numbers were also worth changing. Style findings mattered for readability, but security and correctness issues took priority.

5.3 What changes did you make, and how did you verify them?

I replaced unrestricted expression evaluation with restricted arithmetic handling, replaced pickle storage with JSON validation, used HMAC-SHA-256 for patient identifiers, moved the demo password to an environment variable, removed shell-based label output, and used secrets for prescription-number generation. I also corrected exception handling, mutable defaults, naming, documentation, and formatting. The recorded final results were Pylint 10.00/10, Bandit 0 findings, and Flake8 0 violations. I ran the program and checked the stock report, billing, dosage, login, and stock-update demonstration.

5.4 What challenges did you face, and what would you improve next?

A challenge was balancing security improvements with preserving the program's existing behaviour. Replacing a risky implementation must not silently break normal workflows, so I tested the demonstration after changing the code. Another challenge was keeping baseline and final reports separate and maintaining a clear Git history. Next, I would add automated tests for invalid formulas, malformed JSON stock files, missing environment variables, expired products, and boundary values. I would also test on a clean environment and regenerate all reports from the exact final commit.

Submission note: The exact four reflection questions from the course handout were not available while preparing this document. Compare the headings above with the handout and adjust them if the instructor specified different wording.